Data: Exploration and Preparation

Lesson 2 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

2 October 2026 · reading time about 80 minutes

Contents

Lesson plan

Time	Segment	Material
0:00–0:10	Recap of Exercise 1, today's map	Slides 1–6
0:10–0:35	Exploratory analysis and missing values	Slides 7–19
0:35–1:00	Notebook 01, live	Notebook 01
1:00–1:15	**Break**	
1:15–1:40	Scaling and encoding	Slides 20–31
1:40–2:05	Notebook 02, live	Notebook 02
2:05–2:20	Leakage in preprocessing	Slides 32–41
2:20–2:50	Notebook 03, live	Notebook 03
2:50–3:00	Homework set, questions	Slides 50–51
	Total	**180 minutes**

---

1. From "nothing is learned before the split" to everything else

Exercise 1 is due today. Before starting anything new, it is worth naming what that exercise actually tested: not whether your model scored well, but whether you split the data before touching it and kept the scaler inside a pipeline. Every idea in today's lesson is that same rule, applied to operations you have not met yet.

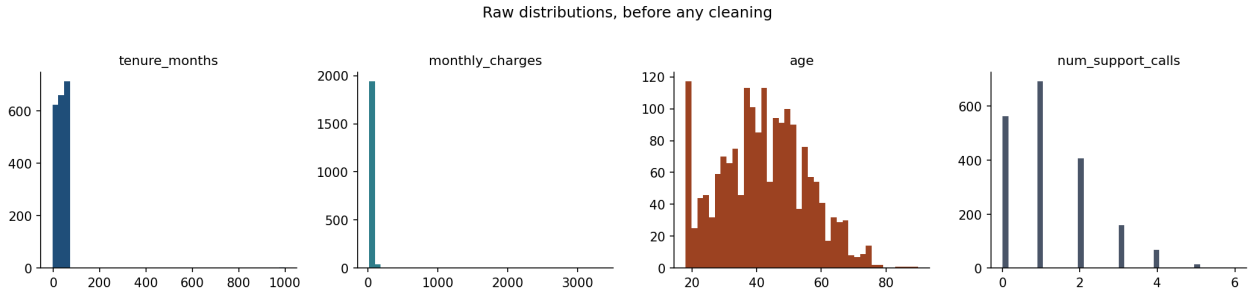
Lesson 1 established one central fact — the test set exists to give an unbiased estimate of $R(f)$, and that estimate stays unbiased only if f was chosen without ever consulting the test rows — and one central tool for enforcing it: the `Pipeline`, which fits `StandardScaler` on training data alone because it sits inside a `fit()` call restricted to the training fold. Today the tool becomes the main character. Real datasets rarely arrive as clean numeric matrices ready for `StandardScaler`. They arrive with gaps, errors, mismatched units, and categories with no numeric meaning, and every one of those has to be repaired *before* a model can be fitted at all. The question this lesson keeps asking is: which repairs *learn something from the data*, and therefore belong inside the pipeline exactly like scaling did — and which merely apply a fixed rule and are safe to do once, up front?

Get that classification right and the discipline from Lesson 1 extends for free. Get it wrong — impute a missing value, or encode a category, using statistics computed over the whole dataset — and you have leaked exactly as surely as if you had scaled before splitting, except this time nothing about the code looks unusual. That is the running example of Sections 3, 6 and 9, and it is the reason this lesson exists in the middle of the course rather than as a footnote to Lesson 1.

Throughout, we use one running dataset: 2000 telecom customers, and whether each one churned (cancelled) in the following month. It is generated by `Notebooks/churn_data.py` rather than downloaded, which buys us something no real dataset offers — we know, by construction, exactly which columns are genuinely predictive, which are pure noise, and which missing values are missing completely at random, missing at random, or would be missing not at random — the three mechanisms named in Section 3.1. Use that knowledge to check your intuition against the ground truth in Sections 3 and 9; real projects rarely afford the check.

---

2. Exploratory analysis



Every numeric column, before anything is done to it. Shapes, ranges and outliers are all visible here and in none of the summary statistics.

Before any transformation, look. This is not a formality; it is the step that tells you which of the remaining sections apply at all, and skipping it is how a `999`-month tenure or a category with one member ends up inside a model undetected.

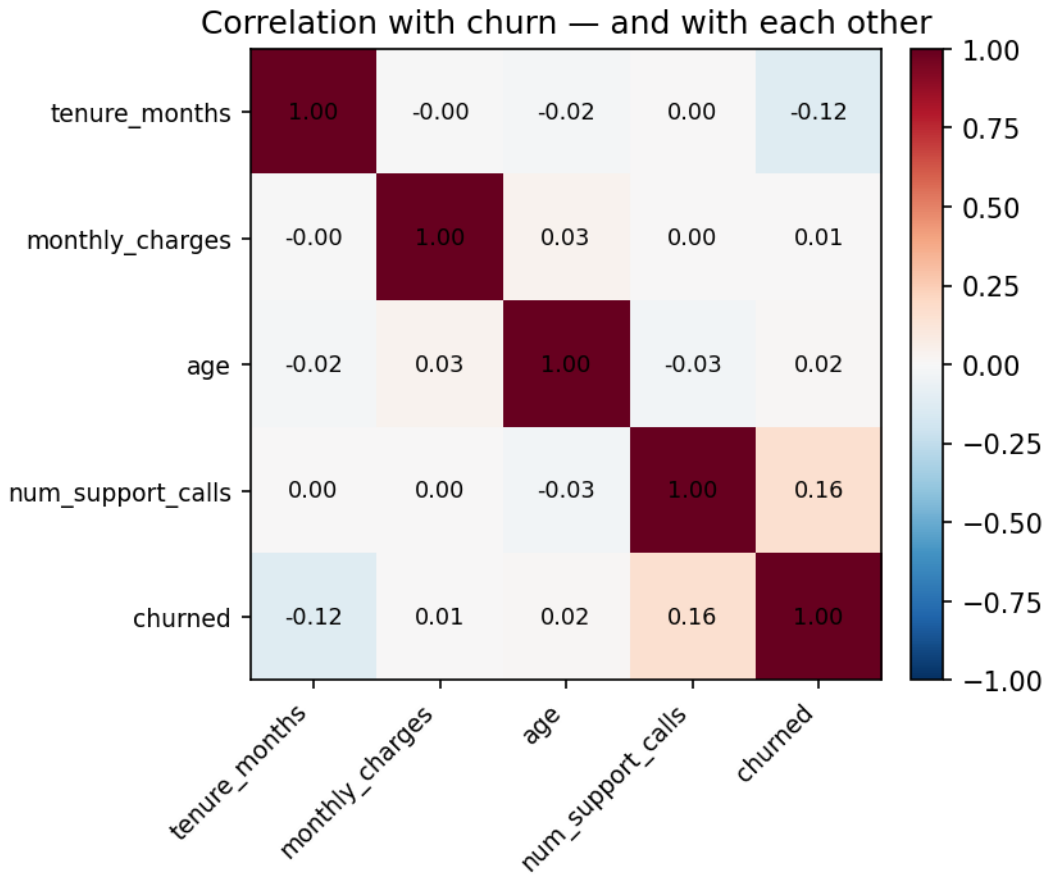
A systematic first pass answers four questions, in this order, and each answer changes what you do next.

How big is it, and what type is each column? `.shape` and `.dtypes` (or `.info()`, which gives both at once) tell you whether you are dealing with thousands of rows or millions, and whether a column pandas has read as an object is actually numeric-with-typos, a genuine category, or free text. A column of "35", "42", "unknown" is not numeric until "unknown" has been decided about — silently coercing it drops those rows without telling you.

What is the target, and how is it distributed? For classification, the class balance sets the baseline every later score is measured against, exactly as in Lesson 1. Our running example churns at 19.4%: a model that never looks at the data and always predicts "no churn" is already right 80.6% of the time, and that number is the one every later accuracy has to beat by a meaningful margin, not a decimal point.

Where are the gaps? `.isna().sum()` per column, covered in full in Section 3.

What is the shape of each numeric column, and how do columns relate to each other and to the target? Histograms surface skew; a correlation matrix surfaces which numeric columns move together and, tentatively, which look related to the target. "Tentatively" is doing real work in that sentence — a correlation is a starting hypothesis, not a conclusion, and Section 9 shows a case where it is actively misleading.

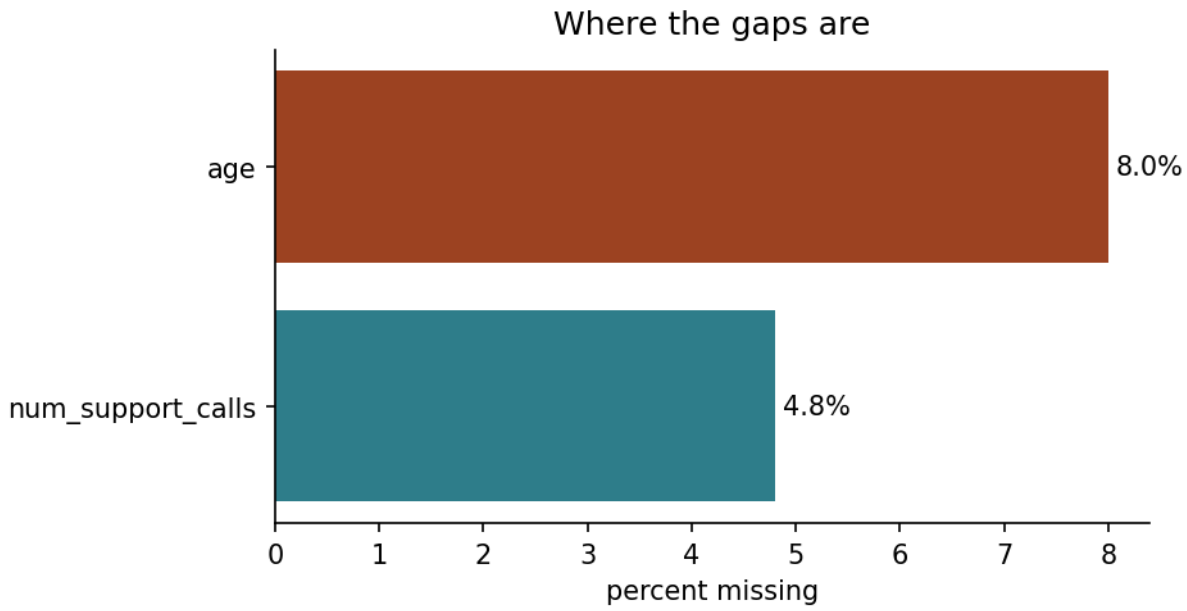


The four numeric columns against each other and against the target. What to look at is how weak the strongest relationship is: `num_support_calls` *correlates with churn at +0.16 and* `tenure_months` *at −0.12, and everything else sits within 0.03 of zero. No single column predicts churn on its own, which is why the rest of this chapter exists — and why Section 9's leaked feature, at a correlation far above anything here, should be read as an alarm rather than a discovery.*

None of this fits a model or learns a parameter, which is exactly why it is safe to do on the *whole* dataset, including the portion that will later become the test set — looking is not fitting. The line you must not cross is using anything you see here to make a decision that changes how the model is built, before splitting. Reading that `monthly_charges` is skewed and deciding, on that basis alone, that a log transform is worth trying, is exploratory; computing the exact transform's parameters from the full dataset and applying them everywhere is not. Section 8 makes the boundary precise.

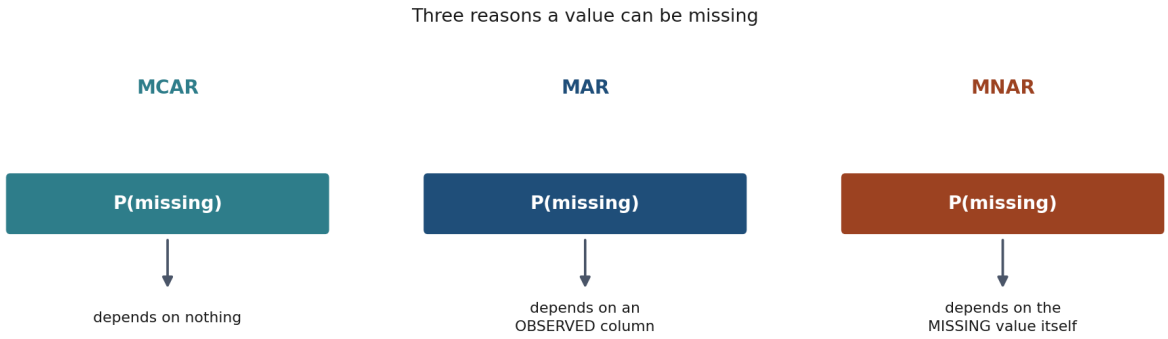
---

3. Missing values



Where the gaps are. What a bar chart of missingness cannot show is whether the gaps are related to each other or to the target — which is exactly what decides the fix.

3.1 Three mechanisms, and why the difference matters



The three mechanisms side by side. The distinction is not academic: it determines which repair is valid and which quietly reweights your sample.

Not all missingness is the same, and treating it as if it were is the most common preparation mistake in practice. The standard taxonomy, due to Rubin (1976), distinguishes three mechanisms by *what the probability of being missing depends on.*

Missing completely at random (MCAR). The probability that x_j is missing for row i does not depend on any value, observed or not:

$$P(\text{missing}_{ij} \mid x_i) = P(\text{missing}_{ij})$$

A form left blank for no systematic reason is MCAR. Under MCAR, the observed rows are a random

subsample of the full data, so simple imputation (the mean, the median) introduces no bias in the estimate of the column's own mean — it does, however, still distort other things, as Section 3.2 derives.

Missing at random (MAR). The probability of being missing depends on *observed* variables, but not on the missing value itself:

$$P(\text{missing}_{ij} \mid x_i) = P(\text{missing}_{ij} \mid x_{i,\text{obs}})$$

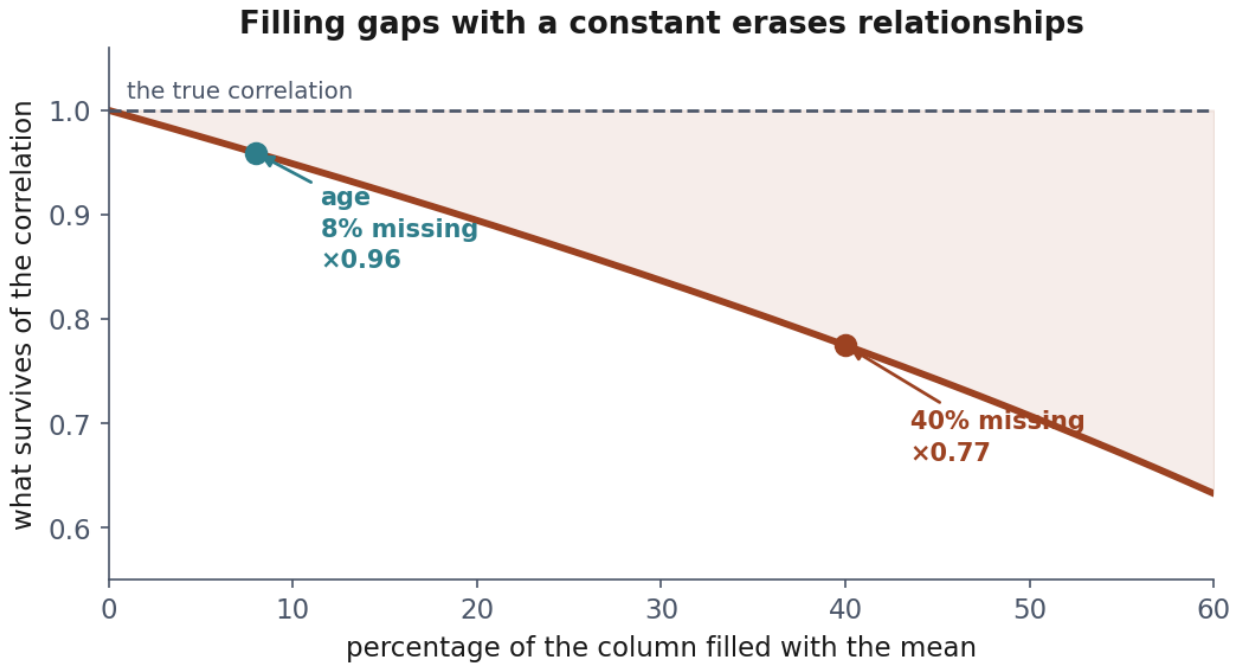
Our `num_support_calls` column is MAR: it goes missing more often for long-tenure customers, because their early call records predate the current CRM system. Conditional on `tenure_months`, which we observe, the missingness carries no further information — but ignoring tenure and treating the gaps as MCAR would be wrong, because the missing rows are *not* a random subsample of all rows; they are disproportionately long-tenure ones.

Missing not at random (MNAR). The probability of being missing depends on the missing value itself, even after conditioning on everything observed:

$$P(\text{missing}_{ij} \mid x_i) \neq P(\text{missing}_{ij} \mid x_{i,\text{obs}})$$

A customer who cancels because their bill was unexpectedly high, and who never finished a sign-up survey as a result, produces MNAR missingness in any field on that survey: the fact of leaving the field blank is correlated with the very value the field would have recorded. No amount of clever imputation recovers this — the information is not merely hidden, its absence *is* evidence about the value. MNAR is diagnosed from domain knowledge about how the data was collected, never from the data alone, and the honest response is usually to flag the pattern and report the limitation rather than to impute past it.

3.2 Why mean imputation is not "no information lost"



What filling with the mean actually costs. The column keeps its average and loses its spread, so its relationship with everything else is attenuated by $\sqrt{1-p}$ — visible here as a flattening cloud.

The picture first. Suppose you know the heights of a class, and for a few students you write down the class average instead of measuring them. The average height of the class is unchanged — you have not biased it. But the class now looks more uniform than it is: the invented values sit exactly in the middle, adding no spread.

Now ask whether height predicts shoe size. The students you invented contribute nothing to that relationship — their height no longer varies with anything — so the relationship looks weaker than it truly is. **You have not biased the column; you have flattened its connection to everything else.**

The algebra below says exactly how much, and the answer is a clean square root.

It is tempting to treat mean imputation as a neutral placeholder — "we did not know, so we used the average, which is the best single guess." The guess is optimal for the *column's own mean*, but it is not neutral for anything downstream. Here is the argument in full, for a column X observed on a subset and missing (MCAR, so unbiased at this first level) on the rest, and its true (unobserved) correlation with another column Y.

Let X have a fraction p of missing entries, replaced by the sample mean $\bar{X}$ of the observed part. Write the imputed column as

$$X'_i = \begin{cases} X_i & \text{observed} \\ \bar{X} & \text{missing} \end{cases}$$

The variance of X' is smaller than the true variance of X: the imputed entries contribute zero to

the sum of squared deviations from the mean, while a genuine draw from X's distribution would not. Precisely, if $\mathrm{Var}(X) = \sigma_X^2$ and imputation is MCAR,

$$\mathrm{Var}(X') = (1-p)\,\sigma_X^2$$

— the variance shrinks by exactly the missing fraction. Now consider the sample covariance between X' and a fully observed Y. Since the imputed entries all take the constant value $\bar{X}$, they contribute nothing to the covariance sum beyond what a constant contributes (zero, once centred), so

$$\mathrm{Cov}(X', Y) = (1-p)\,\mathrm{Cov}(X, Y)$$

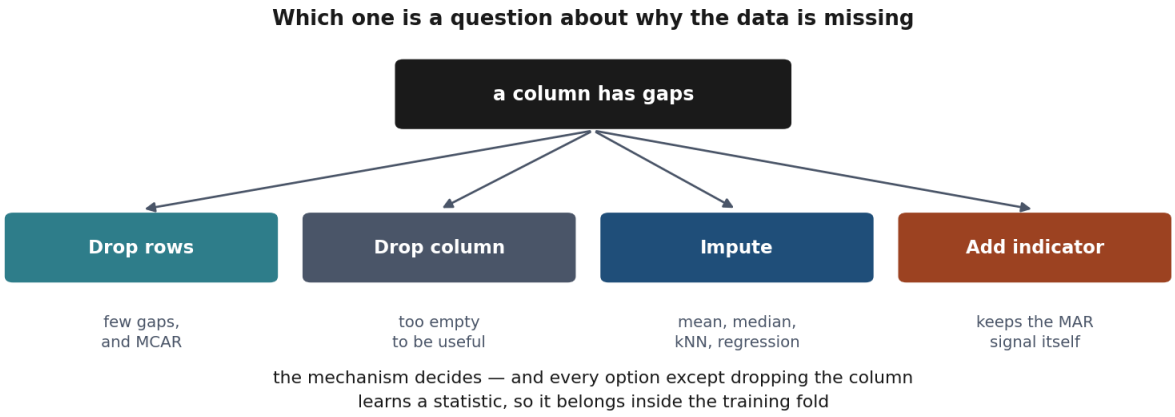
The correlation is the ratio of covariance to the product of standard deviations, and $\mathrm{Var}(X')$ shrank by $(1-p)$ while $\mathrm{Cov}(X', Y)$ also shrank by $(1-p)$ but enters the correlation as a ratio against $\sqrt{\mathrm{Var}(X')}$, not $\mathrm{Var}(X')$ itself:

$$\rho(X', Y) = \frac{(1-p)\,\mathrm{Cov}(X, Y)}{\sqrt{(1-p)\sigma_X^2}\,\sigma_Y} = \sqrt{1-p}\;\rho(X, Y)$$

Mean imputation **attenuates every correlation** the imputed column has with anything else, by a factor of $\sqrt{1-p}$, even when the missingness is perfectly MCAR and the mean itself is unbiased. With $p = 0.08$, as for `age` in our dataset, the attenuation factor is $\sqrt{0.92} \approx 0.96$ — small, and easy to ignore. With $p = 0.4$, it is $\sqrt{0.6} \approx 0.77$: a substantial fraction of a genuine relationship erased by the act of filling gaps with a constant. This is why more elaborate imputers exist — regression imputation, or the nearest-neighbour approach `KNNImputer` uses — which predict the missing value from other columns rather than replacing it with a single constant, preserving more of the covariance structure at the cost of a new place for leakage to enter, which Section 9 returns to.

Where this stops holding. The derivation assumes MCAR missingness and a linear correlation measure; under MAR the attenuation is biased in a direction that depends on which subgroup is missing more, and under MNAR no correction from the observed data alone can fix it — Section 3.1 already said so, and this section is the quantitative version of the same warning.

3.3 What to do about it



The decision, as a chart. Note that every branch depends on the mechanism, which is why Section 3.1 comes first.

- **Drop rows** only when the missing fraction is small and the mechanism is MCAR; otherwise you are silently reweighting the sample.
- **Drop the column** if it is missing too often to be useful and no proxy exists.
- **Impute** — mean or median for numeric columns (median for skewed ones, for the same reason it is preferred for outlier-robust summaries), the most frequent category or a dedicated `"missing"` level for categoricals, `KNNImputer` or a small regression model when the relationship with other columns is worth preserving.
- **Add a missingness indicator.** A binary column recording *whether* a value was missing keeps the MAR signal (the fact that long-tenure customers are more often missing a call count) available to the model even after the gap itself has been filled with a number.

Whichever you choose, Section 8 states the constraint that applies to every option except dropping columns outright: the statistic used to fill a gap — a mean, a median, a set of neighbours — is *learned from data*, and must be learned from the training fold only.

---

4. Outliers

What the word means. An **outlier** is a value that sits far away from the rest of the values in its column — far enough that it looks as though it might not have come from the same process as everything else. That is the whole definition, and notice what it does *not* say: it does not say the value is wrong.

That matters, because a value can be unusual for three quite different reasons, and only the first is a defect:

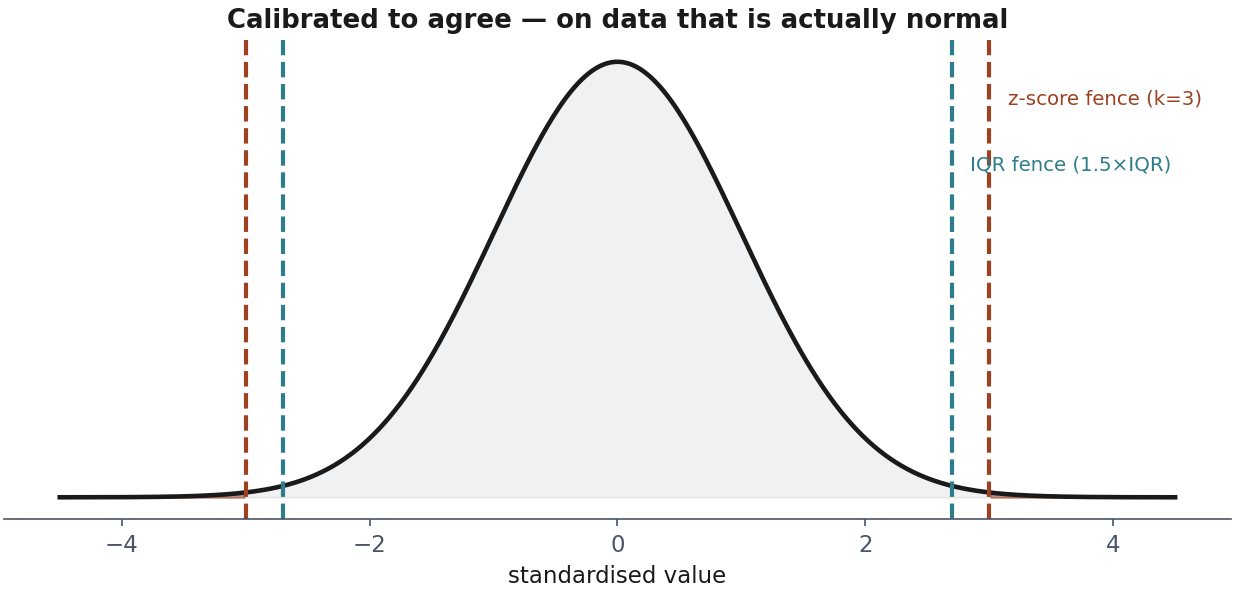
- **A recording error.** In our data, `tenure_months` runs from **−3 to 999** while ordinary customers sit between 0 and 72. Nobody has been a customer for minus three months, and 999 is the shape a "no value here" placeholder takes when somebody types it into a numeric column. Likewise `monthly_charges` reaches **3,344.7** against an ordinary maximum of 128: sixteen rows carry a billing error.
- **A rare but genuine value.** A customer really does pay far more than the others, because they bought everything. Throwing that row away does not clean the data; it deletes a real customer and quietly narrows what the model believes is possible.
- **A value from a different population.** A business account in a dataset of household ones. Real, correctly recorded, and still not what the model is being asked to learn about.

The uncomfortable part: the number alone cannot tell you which. A charge of 3,344.7 looks identical to the arithmetic whether it is a typo or a genuine corporate account. Deciding requires knowing how the data was recorded — which is why this section gives you rules that *flag* candidates and never rules that delete them.

Why bother at all. Because a handful of extreme values changes results out of all proportion to their number. Section 5.1 measures exactly that: sixteen billing errors, out of 1,500 rows, compress every ordinary customer into 3.4% of the min-max range. Nothing was deleted and nothing raised an error; the column was simply ruined for the model that came next.

The two rules below are detectors. What to do once something is flagged is Section 4.2's question, and it has no automatic answer.

4.1 Two rules, derived



On a normal column the two rules very nearly agree — which is no accident, since the constant 1.5 was chosen to make them agree here.

The picture first. Both rules answer the same question — *how far from the middle is too far?* — and differ in what they use as a ruler.

The z-score measures distance in **standard deviations**, so it asks how surprising a value would be if the column were a bell curve. Tukey's rule measures distance in **quartiles**, so it asks how far a value sits beyond the bulk of the data, without assuming any shape.

That difference is why they disagree on skewed data, which is Section 4.2. The constant 1.5 exists to make them agree on data that *is* a bell curve, and the derivation below is where it comes from.

The z-score rule. Standardise and threshold:

$$z_i = \frac{x_i - \bar{x}}{s}, \qquad \text{flag } x_i \text{ if } |z_i| > k$$

Under a normal model this has a direct probabilistic reading: for $X \sim \mathcal{N}(\mu, \sigma^2)$, $P(|Z| > 3) \approx 0.0027$, so $k = 3$ flags roughly the most extreme 0.27% of a genuinely normal column — a principled choice, but one that inherits every assumption of normality, and Section 4.2 shows exactly how it fails.

Tukey's interquartile range (IQR) rule. Let Q_1, Q_3 be the first and third quartiles and $\text{IQR} = Q_3 - Q_1$. Flag x_i outside

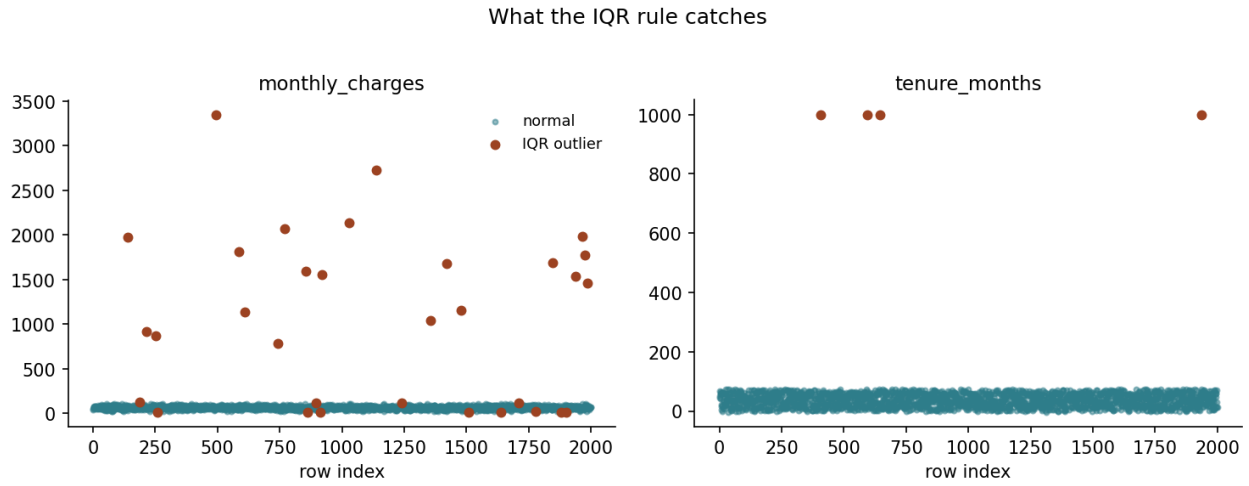
$$[\,Q_1 - 1.5 \cdot \text{IQR},\quad Q_3 + 1.5 \cdot \text{IQR}\,]$$

The constant 1.5 is not arbitrary. For $X \sim \mathcal{N}(\mu, \sigma^2)$, the quartiles are $Q_1 = \mu - 0.6745\sigma$ and $Q_3 = \mu + 0.6745\sigma$ (from the standard normal quantile function $\Phi^{-1}(0.25) \approx -0.6745$), so IQR $= 1.349\sigma$ and the upper fence sits at

$$Q_3 + 1.5 \cdot \text{IQR} = \mu + 0.6745\sigma + 1.5(1.349\sigma) = \mu + 2.698\sigma$$

which flags roughly the most extreme 0.35% on each tail under normality — Tukey chose 1.5 so that this rule and the $k = 3$ z-score rule flag comparable fractions of a genuinely normal column. The two rules are calibrated to agree with each other on well-behaved data, and Section 4.2 is precisely about what happens when the data is not well behaved.

4.2 Why they disagree on real data, and what neither of them can tell you



On a skewed column they part company: Tukey's fence flags a long tail that the z-score rule, which assumes symmetry, leaves alone.

Both $\bar{x}$ and s in the z-score rule are themselves computed from the data, including the outliers — a handful of extreme billing errors inflate s directly, widening the flagging threshold and making the rule *less* sensitive precisely when it should be more so. This is the outlier detection analogue of the leakage argument that runs through the whole lesson: a statistic that is supposed to characterise "normal" is contaminated by the abnormal points it is meant to catch. Q_1 and Q_3 move far less under contamination, because a handful of extreme points cannot shift a quartile unless they make up more than 25% of the data — which is why IQR is the more common default for skewed, error-prone columns.

Neither rule, however, can tell you whether a flagged point is a *mistake*. Both are purely distributional: they find values that are unusual relative to the rest of the column, and say nothing about whether a value is *possible* at all. A tenure of −3 months is impossible on its face, yet if negative tenures make up only a small fraction of a column that already spans a wide range, they are not *extreme* relative to that spread and neither rule flags them — notebook 1 shows this exactly. Catching them needs a **domain rule** (tenure cannot be negative or exceed some sane maximum), which is a different kind of check entirely: not "is this unusual" but "is this valid". A thorough pass runs both, because they catch different mistakes.

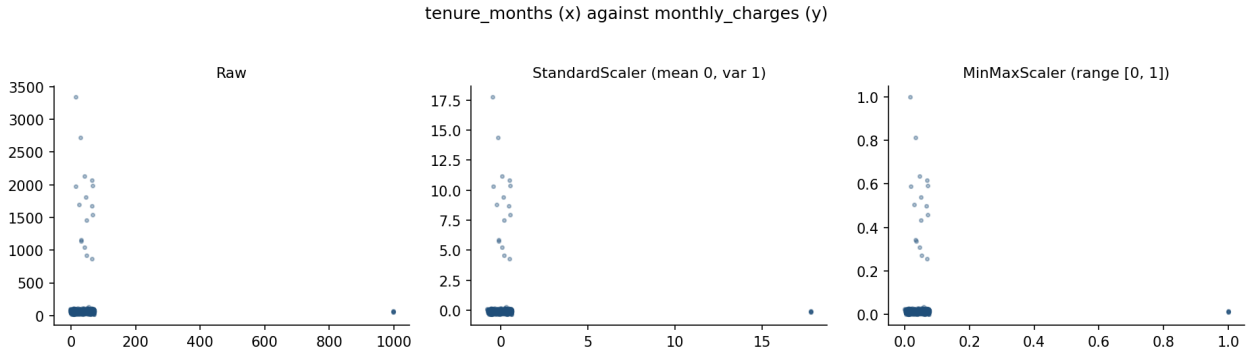
Deciding what to do once something is flagged is a fourth, separate step, and it is a domain decision, not a statistical one: cap it (Winsorise) at the fence, remove the row, correct it if the true value is recoverable, or leave it if it is unusual but genuine. A customer who has stayed 70 months and pays a high bill is not an error; a tenure of 999 months is.

> **Try this:** in notebook 1, lower the z-score threshold from $k = 3$ to $k = 2$ and count how many ordinary customers get swept in with the genuine errors. The threshold is a decision with a cost on both sides.

---

5. Feature scaling

5.1 Two standard transforms



The same column under both transforms. Standardisation centres and rescales by the spread; min-max squeezes into $[0, 1]$ *and is therefore at the mercy of a single extreme value.*

$$z = \frac{x - \mu}{\sigma} \qquad\qquad x_{\text{minmax}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Standardisation centres each feature at 0 with unit variance; min-max scaling maps each feature into $[0, 1]$. Min-max scaling is more sensitive to a single extreme value, since $x_{\max}$ or $x_{\min}$ can be an outlier that compresses every ordinary value into a sliver of the range — another reason outlier handling (Section 4) precedes scaling, not the other way round.

"A sliver" is measurable, so measure it. In the training split, `monthly_charges` runs from 15.0 to 128.4 for the 1,484 ordinary customers and contains 16 billing errors, the largest at 3,344.7. Fit both scalers on that column as it stands and take one perfectly ordinary customer paying 80.0 a month:

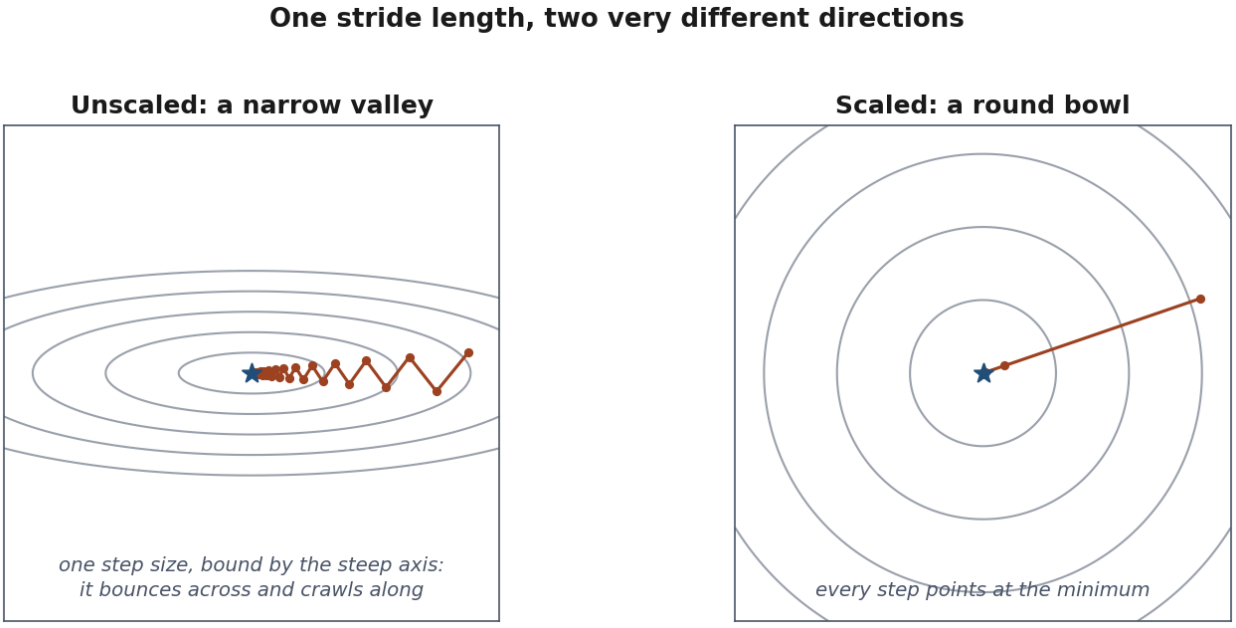
	$x_{\min}$, $x_{\max}$	μ, σ	where 80.0 lands
with the 16 errors	15.0, 3344.7	81.22, 183.65	min-max **0.020** · z **−0.007**
without them	15.0, 128.4	63.55, 17.07	min-max **0.573** · z **+0.964**

Under min-max, every ordinary customer is now packed into $[0.000,\ 0.034]$ — **3.4% of the range**, with the other 96.6% reserved for sixteen rows. The sliver is not a metaphor.

Standardisation is pulled in the same direction, and it is worth being precise about how much rather than waving at it: the spread it divides by grows from 17.07 to 183.65, a factor of **10.8**, while min-max's denominator grows from 113.4 to 3329.7, a factor of **29.4**. So min-max is about **three times** as badly affected here — a real difference, but not the difference in kind that "more sensitive" can suggest. Neither transform repairs an outlier. Only Section 4 does.

Scaling matters for two distinct families of method, for two different reasons. **Distance-based methods** (k-nearest neighbours, k-means, anything using a Euclidean or similar metric, all met from Lesson 6 onward) compute distances as sums of per-feature squared differences; an unscaled feature with a numerically larger range dominates that sum regardless of how informative it actually is. **Gradient-based methods** (linear and logistic regression fitted by gradient descent, and every neural network in Lessons 9 and 10) are affected for a sharper, more precise reason, derived in full below.

5.2 Why it matters for gradient descent



The cost surface when two features differ in variance. It is not a bowl but a ravine — steep across, almost flat along — and one stride length has to serve both directions.

The picture first. Think of the cost as a landscape you are descending, and of each feature as one compass direction. If one feature varies over thousands and another over units, the landscape is not a bowl but a **narrow ravine**: steep across, almost flat along.

You have one stride length for both directions. Make it long enough to make progress along the flat axis and you overshoot the steep one, bouncing from wall to wall. Make it short enough to be safe on the steep axis and you crawl along the flat one.

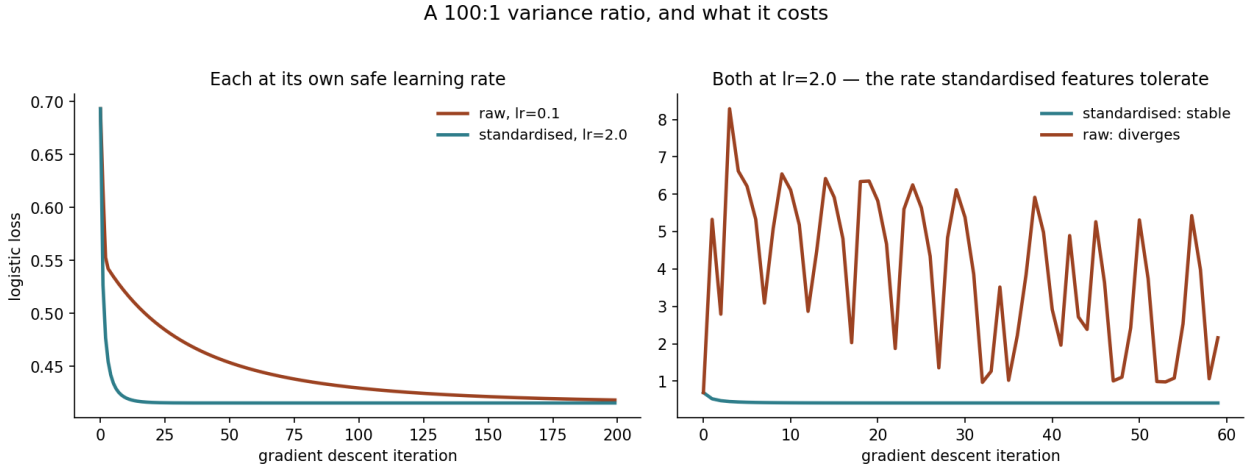
Scaling reshapes the ravine into a bowl, so a single stride length suits every direction.

What follows from that, stated rather than derived. Most models in this course are fitted by **gradient descent**: start somewhere on the landscape and take repeated steps downhill, every step the same size. Two consequences follow from the picture alone.

- **The steepest direction sets the pace for all of them.** A stride that is safe on the steep wall is the largest you may take, because a longer one overshoots and the cost goes *up*. So the feature with the largest spread decides the step size, however uninformative that feature happens to be.
- **At that pace the flat direction barely moves.** How many steps you need grows with how stretched the ravine is — the ratio of the largest feature variance to the smallest, which has a name, the **condition number**.

Standardising sets every variance to 1. The ravine becomes a bowl, the ratio falls to roughly 1, and one stride suits every direction at once.

On the data. Notebook 2 builds two features with a variance ratio of 110 — standard deviations of 0.98 and 10.23 — and fits the same model twice. The scaled version is stable at a learning rate of 2.0; the raw one needs 0.1, a rate **twenty times smaller**, and after 200 steps has reached a cost of 0.4183 against the scaled run's 0.4155. Give the raw features the scaled version's learning rate and it does not merely converge slowly: the cost reaches **3.34**, far above where it started. It diverges.



The same problem, scaled and unscaled, at the same learning rate. The unscaled run is still travelling when the scaled one has arrived — same data, same model, same number of steps in both panels.

That is the whole practical content: **scaling is not cosmetic. It changes how long training takes, and sometimes whether it finishes at all.**

Where the details live. Why the shape of that landscape is fixed by the feature covariances, and why the safe stride is exactly two over the largest variance, belongs with the derivation of gradient descent itself — Lesson 3, whose Section 4.4 computes the condition number on the real housing design matrix and finds that standardising takes it from 285 to 3.4, a factor of about 80. Nothing here depends on those steps; if you want them now, they are waiting there.

Two caveats worth carrying forward. The argument is about *gradient descent with one global step size*; optimisers that adapt a rate per parameter — Adam, met in Lesson 9 — are markedly less sensitive to this, though scaling still rarely hurts and remains the default. And the exact statement is for linear regression's squared-error cost; for logistic regression the landscape changes shape as the model moves, but features with very different spreads stretch it the same way, so the conclusion carries.

Try this: in notebook 2, change the toy example's variance ratio from 100 to 9 and

to 900, and note how the gap between the two learning rates that stay stable grows or shrinks. It should track the ratio directly.

---

6. Categorical encoding

A model does arithmetic. A column holding `"month-to-month"` offers nothing to do arithmetic with, so it has to become numbers — and **every way of doing that makes a claim about the categories**. The claim is the part to get right; the code is three lines either way.

Take `contract_type`, which has three levels and 2,000 rows behind it:

	rows	churn rate
`month-to-month`	1,092	0.266
`one-year`	488	0.137
`two-year`	420	0.071

Here is what one customer on a one-year contract becomes under each encoding, and what each is asserting:

Encoding	That customer becomes	The claim being made
One-hot	`[0, 1, 0]`	the three levels are simply different; no order, no distances
Ordinal	`1`	they are ordered, **and** the gap from month-to-month to one-year equals the gap from one-year to two-year
Target	`0.137`	the level's average outcome summarises it well enough to stand in for it

The middle row is where damage usually happens, and this column shows why. The order is genuine — a one-year contract really does sit between the other two — so ordinal encoding looks safe. But the churn rate falls by 0.129 on the first step and by only 0.066 on the second: the first gap is **twice** the second, while the encoding $0, 1, 2$ tells the model they are the same size. A linear model given that column fits one coefficient for a step whose meaning changes depending on where you take it.

The three subsections take the three encodings in turn.

6.1 One-hot encoding, and the dummy variable trap, proved

Why all k dummies plus an intercept cannot work

contract_type	m2m	1y	2y	sum	intercept
month-to-month	**1**	0	0	**= 1**	**1**
one-year	0	**1**	0	**= 1**	**1**
two-year	0	0	**1**	**= 1**	**1**

the k dummy columns always sum to the intercept column — rank deficient by exactly one

The redundancy made visible: the dummy columns sum to the intercept column, so the design matrix loses rank by exactly one and the solution stops being unique.

The picture first. Suppose a survey asks whether you travel by car, bus or bicycle, and you record three yes/no columns. Those columns carry a hidden redundancy: knowing two of them tells you the third, because everyone answered exactly one.

Add an intercept — a column of ones — and the redundancy becomes exact: the three dummies always sum to that column. The model can now shift value between the intercept and the dummies without changing a single prediction, so there is no unique best answer, only infinitely many equally good ones.

Dropping one category fixes it by removing the spare degree of freedom. The proof below is that paragraph in matrix form.

A categorical column with k levels becomes k binary columns, one per level, each row having a single 1. If a model fits an intercept term — a constant column of 1s — alongside *all* k dummy columns, the design matrix becomes singular. The proof is one line: for any row i,

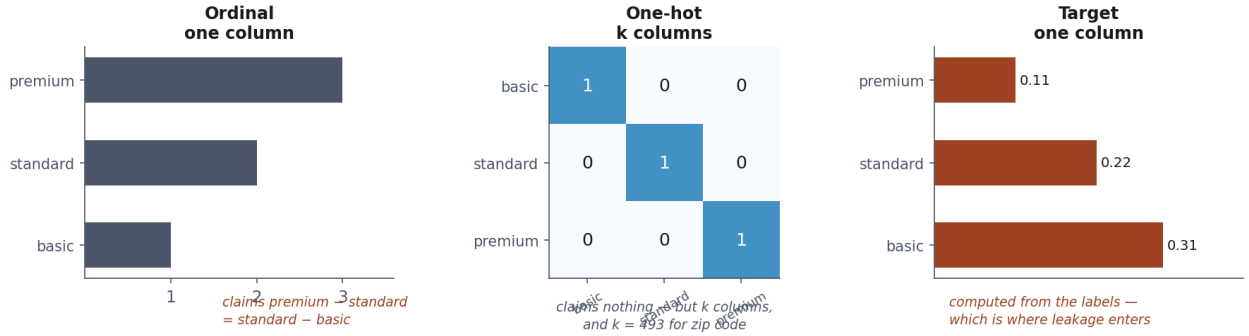
$$\sum_{j=1}^{k} \text{dummy}_j^{(i)} = 1 = \text{intercept}^{(i)}$$

every row's dummy columns sum to exactly 1, which is precisely the intercept column. The intercept column is therefore a linear combination of the k dummy columns (their sum), so the $k+1$ columns together have rank at most k, not $k+1$: **the design matrix is rank-deficient by exactly one.** Notebook 2 confirms this directly with `numpy.linalg.matrix_rank`. A singular or near-singular design matrix means the normal equations $(X^\top X)^{-1}X^\top y$ have no unique solution — infinitely many coefficient vectors fit the training data identically, and which one a solver returns becomes an artefact of numerical noise rather than a meaningful estimate.

The fix is to drop one level, encoding k categories with $k-1$ columns; the dropped level becomes the *reference category*, absorbed into the intercept. This loses no information — the k-th category is

recoverable as "all other dummies are 0" — and restores full rank. `OneHotEncoder(drop="first")` implements this directly. Models that do not fit an intercept, and tree-based models met in Lesson 7, which split on one dummy at a time and do not care about collinearity, can safely use all k columns.

6.2 Ordinal and target encoding

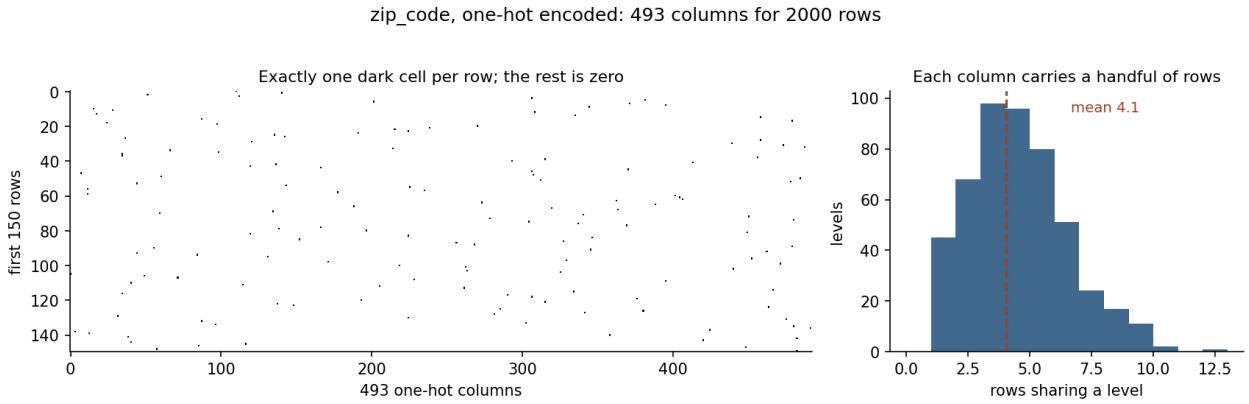


Three encodings of the same column, each making a different claim about the categories — that they are unordered, that they are ordered, or that they can be summarised by their average outcome.

Ordinal encoding maps categories to integers, $\{0, 1, \ldots, k-1\}$. It is appropriate only when the categories have a genuine order (`"low"`, `"medium"`, `"high"`) — imposing an arbitrary integer order on an unordered category (encoding `"North"` as 0 and `"South"` as 1) tells a model that North and South are numerically closer than North and West, which is meaningless and actively misleading for any method that uses that number arithmetically.

Target encoding replaces a category with a statistic of the target computed over the rows sharing it — typically the mean, $\bar{y}_c$ for category c. On `contract_type` that is the third column of the table above — 0.266, 0.137, 0.071 — and the appeal is immediate: three levels have become one column that already carries most of what the levels were telling us. It compresses arbitrarily many levels into a single, often highly predictive, numeric column, which is exactly why it exists for `zip_code`-shaped columns where one-hot encoding would add hundreds of sparse columns. It is also, computed the obvious way, the more dangerous of the two encodings in this entire lesson, and Section 9.2 derives exactly why.

6.3 The curse of dimensionality, briefly



What one-hot encoding `zip_code` actually produces, drawn from the data itself. Left: 493 columns, one dark cell per row and nothing else — 99.8% of the matrix is zero. Right: the number of rows sharing a level, averaging 4.1. A column that carries four rows cannot support a coefficient estimated from those four rows, and that is the curse of dimensionality in its most concrete form.

One-hot encoding a column with k levels adds k columns, most of which are zero for most rows. Beyond a modest k this has two costs: the feature space grows large relative to the number of examples, so distance-based methods (Lesson 6) find every point roughly equidistant from every other, and the number of parameters a model must estimate grows with it, increasing variance for a fixed sample size. `zip_code`, with 493 levels across 2000 rows, sits squarely in the range where one-hot encoding is impractical and target encoding — done correctly — is the standard alternative.

---

7. Feature engineering

A model can only use the information present in its input columns, in the form it is presented. Linear and logistic regression, in particular, can only represent relationships that are linear in the features they are given — they cannot discover, on their own, that the *ratio* of two columns matters more than either alone, unless that ratio is computed and handed to them as a new column.

Three common constructions:

- **Ratios and differences**, when a rate matters more than either raw quantity — `monthly_charges` relative to `tenure_months` captures "cost intensity" in a way neither column does alone.
- **Interaction terms**, $x_j \cdot x_k$, when the *combination* of two features matters beyond their individual effects — a model with only `contract_type` and `monthly_charges` cannot represent "high charges matter more for month-to-month customers" unless that product is added explicitly.
- **Binning**, converting a continuous variable into ordered categories, which lets a linear model approximate a non-linear (e.g. non-monotonic) relationship with the target piecewise, at the cost of discarding within-bin information.

Feature engineering is a hypothesis about the domain, not a guaranteed improvement. Notebook 2's engineered `charge_per_tenure` moves the model's area under the receiver operating characteristic curve (AUC) from **0.752 to 0.754** — two thousandths.

(AUC appears repeatedly from here on, so fix it now: it is the probability that the model gives a randomly chosen churner a higher score than a randomly chosen non-churner. A coin flip scores 0.5, a perfect ranking 1.0, and unlike accuracy it does not change when the classes are imbalanced. Lesson 4 derives it properly; until then, read it as "how well does this rank the two groups apart".) Worth stating plainly: that is **not** a success, and reporting it as one would be the first step towards the reasoning Lesson 5 exists to prevent. It is a legitimate result — the hypothesis that cost intensity matters here was reasonable, it was tested, and the data declined it — and it is also within the range that a different train/test split could produce on its own, which is precisely why Lesson 5 will not let a single split settle a question this small.

The three constructions are worth seeing on the actual columns:

Construction	On this dataset	What it lets a linear model express
Ratio	`monthly_charges / tenure_months`	cost intensity — a customer paying 90 for three months is not the customer paying 90 for sixty
Interaction	`monthly_charges ×` `[contract is month-to-month]`	that a high bill matters more when there is nothing holding the customer in place
Binning	`tenure_months` into 0–6, 6–24, 24+	that risk is high early, flattens, then flattens again — a shape no single coefficient on `tenure_months` can produce

statistic learned from data — bin edges chosen from quantiles, an interaction scaled by a learned coefficient — Section 8's rule applies to it exactly as it applies to imputation and scaling.

---

8. The pipeline, generalised

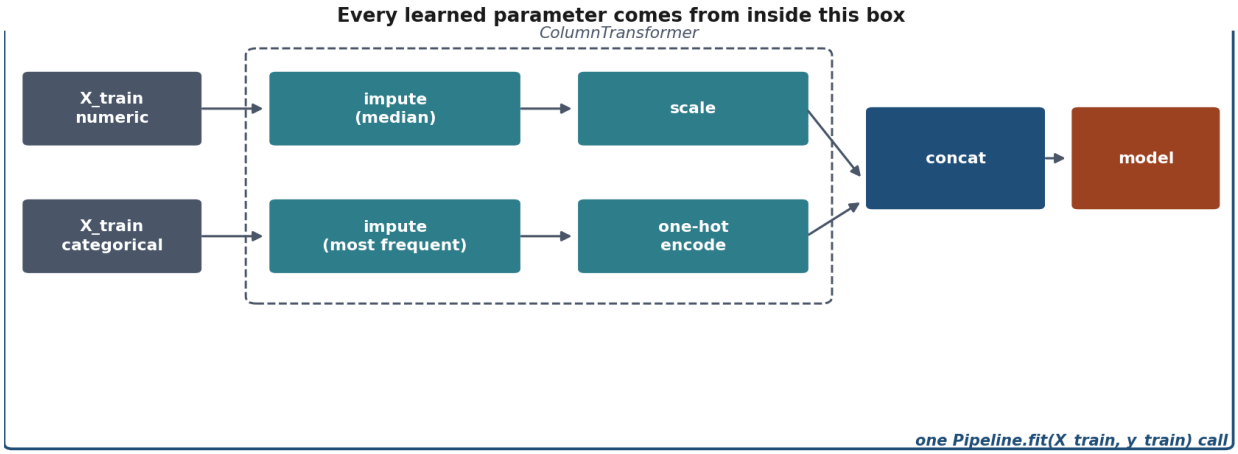
8.1 Restating Lesson 1's argument for any preprocessing step

Lesson 1 showed that a test set T, independent of the fitted function f, gives an unbiased estimate of the expected risk:

$$\mathbb{E}_{T\sim\mathcal{D}^m}\left[\hat{R}_T(f)\right] = R(f)$$

Nothing in that argument mentions "the model." f is whatever function maps raw inputs to predictions, and if preprocessing is baked into that function — which it always is, in deployment, since a fitted scaler or imputer travels with the model — then f includes every preprocessing step's learned parameters: a mean, a set of neighbours, a per-category average. The independence condition applies to f as a whole, not to "the model part" of it. The moment any of those parameters is estimated using rows that later appear in T, f is no longer independent of T, the equality above fails, and $\hat{R}_T(f)$ becomes an optimistic estimate by an amount that has no simple formula — it depends on how much information leaked and through which channel. Sections 3.2 and 9.2 quantify two specific channels; there is no guarantee a third one is not lurking in a preprocessing step this lesson did not name.

8.2 ColumnTransformer and Pipeline



The architecture that makes the rule enforceable rather than remembered: numeric and categorical branches, each fitted on training rows only, joined into one object that can be cross-validated whole.

Different columns need different treatment — numeric columns need imputing then scaling, categorical columns need imputing then encoding. `ColumnTransformer` applies a distinct sub-pipeline to each named group of columns and concatenates the results into a single feature matrix:

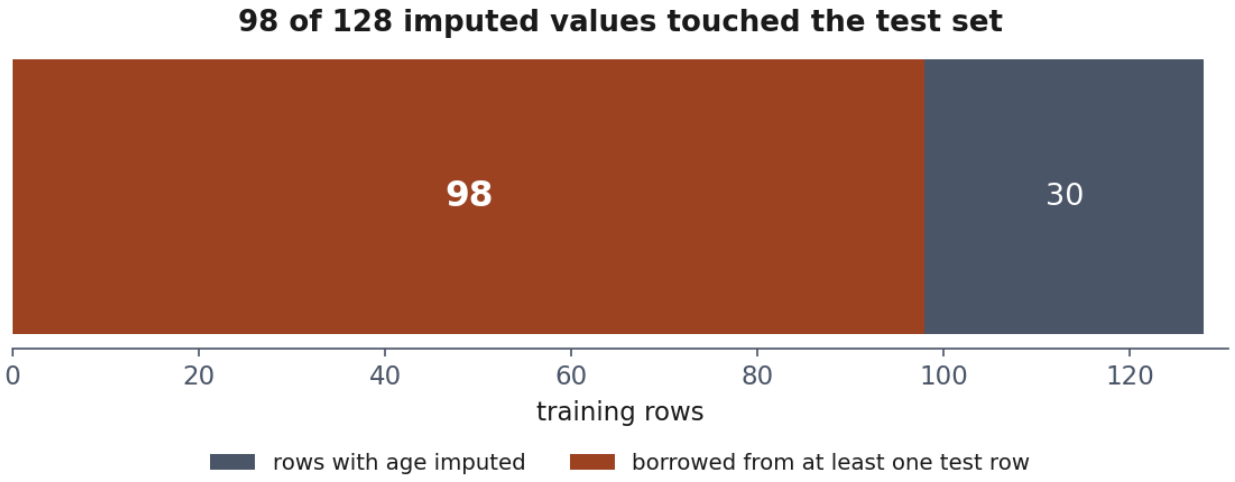
```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])
categorical_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(drop="first", handle_unknown="ignore")),
])
preprocess = ColumnTransformer([
    ("numeric", numeric_pipe, numeric_columns),
    ("categorical", categorical_pipe, categorical_columns),
])
model = Pipeline([("preprocess", preprocess), ("classifier", LogisticRegression())])
model.fit(X_train, y_train)
```

A single call to `model.fit(X_train, y_train)` fits every imputer, the scaler, the encoder and the classifier, **in that order, on `X_train` alone**. No step can see `X_test` during fitting, because no step is ever called on it until `model.predict(X_test)`, by which point every learned parameter is already fixed. This is what Section 8.1 demands, enforced by the object's structure rather than by the discipline of whoever writes the script — which matters, because Section 9 is about exactly the situations where discipline alone silently fails.

Try this: build the same preprocessing by hand — fit a `SimpleImputer` and a `StandardScaler` as separate objects on `X_train`, then apply them to both `X_train` and `X_test` — and check the predictions match the pipeline version exactly. They should; the pipeline changes nothing about what happens, only what can accidentally go wrong.

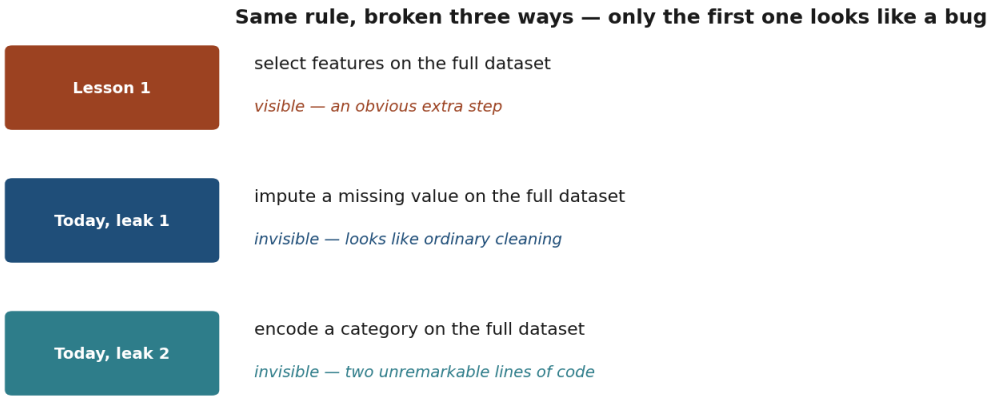
---

9. Data leakage in preprocessing



The evidence that something has leaked: a feature that should carry nothing, carrying a great deal.

9.1 The same rule, two invisible violations



Three ways to break one rule. None of them raises an error, and all three produce a score that is better than the truth.

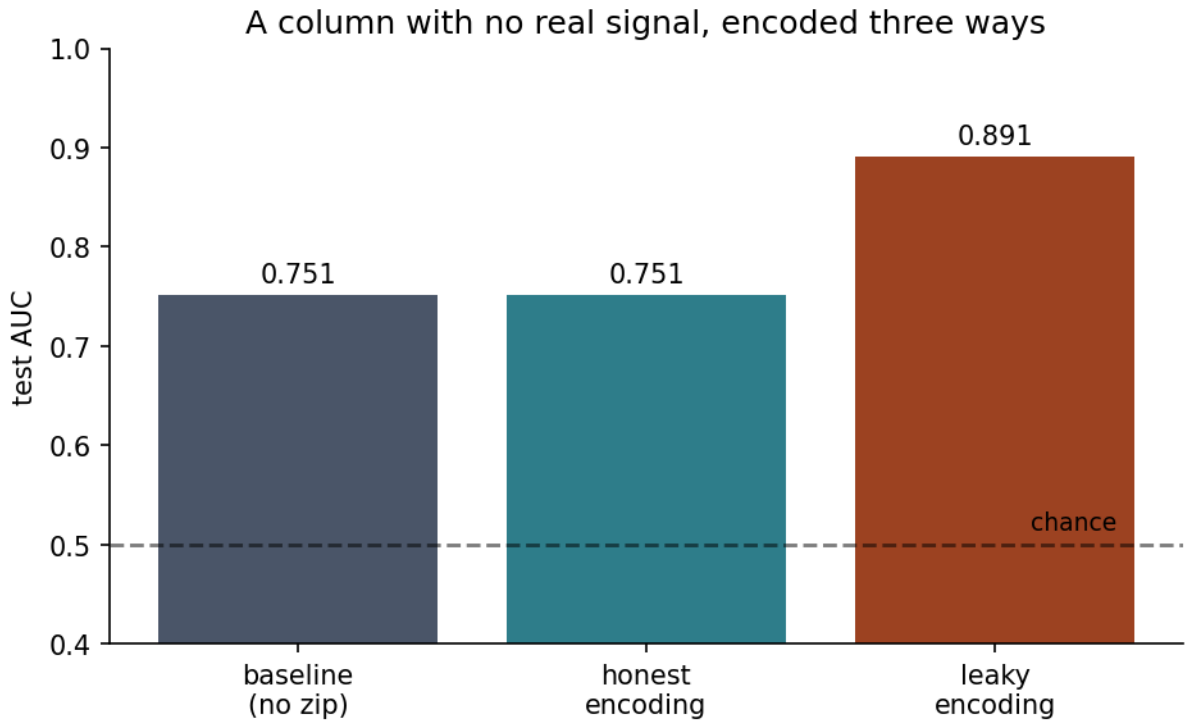
Lesson 1's leakage demonstration selected features using the whole dataset before splitting — visible, once you know to look, because feature selection is obviously "using the data." Imputation and

encoding are more dangerous precisely because they do not read as "training a model" at all: filling in a missing age or replacing a zip code with a number feels like data cleaning, a preparatory chore, not a learning step. It is a learning step. `KNNImputer` learns a neighbour structure; target encoding learns a per-category average. Both are statistics estimated from data, and Section 8.1's argument applies to them without modification.

Imputation. A `KNNImputer` (or any imputer that uses other rows' values, rather than a fixed global statistic) fitted on the concatenation of training and test data finds neighbours anywhere in that concatenation. A training row's missing value can be filled in using a *test* row's observed value. Notebook 3 traces this directly: of 128 training rows with a missing `age`, 98 had at least one test-set row among the five nearest neighbours used to impute it. Nothing raised an error. The measured effect on the final model's test AUC in that run was small (0.7548 versus 0.7530) — worth sitting with rather than dismissing, because a leak does not have to be dramatic to be an optimistic number reported to whoever reads it next, and a dataset where the affected column mattered more, or where more rows were affected, would show a substantially larger gap through exactly the same mechanism.

Encoding. This is the more dramatic of the two, and Section 9.2 derives why.

9.2 Why target encoding leaks, and why small groups make it worse



A column with no real signal, encoded three ways. Target encoding manufactures a predictor out of the labels themselves.

The picture first. Target encoding replaces a category with the average outcome of the rows in it. So ask what happens to a category containing exactly one row: its average *is* that row's own label, copied into a feature under a different name.

Said in one sentence, and this is the whole mechanism:

Your own label gets a vote in your own feature, and the fewer people share your category, the bigger your vote.

With a category of one you are the only voter, so the feature *is* the label. With fifty, your vote is 2% and the column is a genuine statistic about the group. The size of the leak is therefore not a constant — it is one over the size of your group:

rows sharing your category	how much of your own feature is your own label
1	**100%** — the feature is the label
2	50%
5	20%
50	2%
500	0.2%

Worked, on five customers. Take a category with 5 rows, 2 of whom churned, and encode it leakily — averaging over everyone, yourself included. Every row in that category gets the feature $2/5 = \mathbf{0.40}$. Now ask what each row *should* have received, if its own label had been left out:

the row	others in the category	honest value	leaky value	pushed
a churner	1 of the other 4 churned	$1/4 = 0.25$	0.40	**+0.15 up**
a non-churner	2 of the other 4 churned	$2/4 = 0.50$	0.40	**−0.10 down**

Read the last column, because it is the point. The leak is not a vague optimism sprinkled evenly over the data: **churners are pushed up and non-churners are pushed down, every time, in every category.** The encoded column separates the two classes by construction — not because it knows anything about them, but because each row's own answer was mixed into its own question. A model then discovers a "predictor" that is a slightly blurred copy of the labels.

That is why the number in Section 9.1 is what it is. On our churn data `zip_code` has, by construction, no relationship with churn at all — Section 2's correlation check already showed that. Encoded leakily, before the split, it still drives test AUC from **0.751** for a model without it to **0.891**. Nothing was discovered. The labels were fed back in.

Why high-cardinality columns are the dangerous ones. More levels means fewer rows per level, and the table above says the leak grows as the groups shrink. `zip_code` averages **4.1 rows per level** (Section 6.3), so a typical customer's own label makes up roughly a quarter of that customer's own feature. Notebook 3 finds the extreme case and prints it: zip code `Z378` has exactly one customer, is encoded as `1.000`, and that customer's true `churned` value is `1`. The feature and the answer are the same number.

The fix follows from the sentence. Never let a row's own label vote on its own encoded value. `sklearn.preprocessing.TargetEncoder` does this by cross-fitting: each training row's encoded value is computed from the *other* folds of the training data, which is "leave yourself out" done a fold at a time for speed. Used inside a `Pipeline` fitted on `X_train` alone, it recovers an AUC of **0.752**

— indistinguishable from not using `zip_code` at all, which is the correct answer for a column with nothing in it.

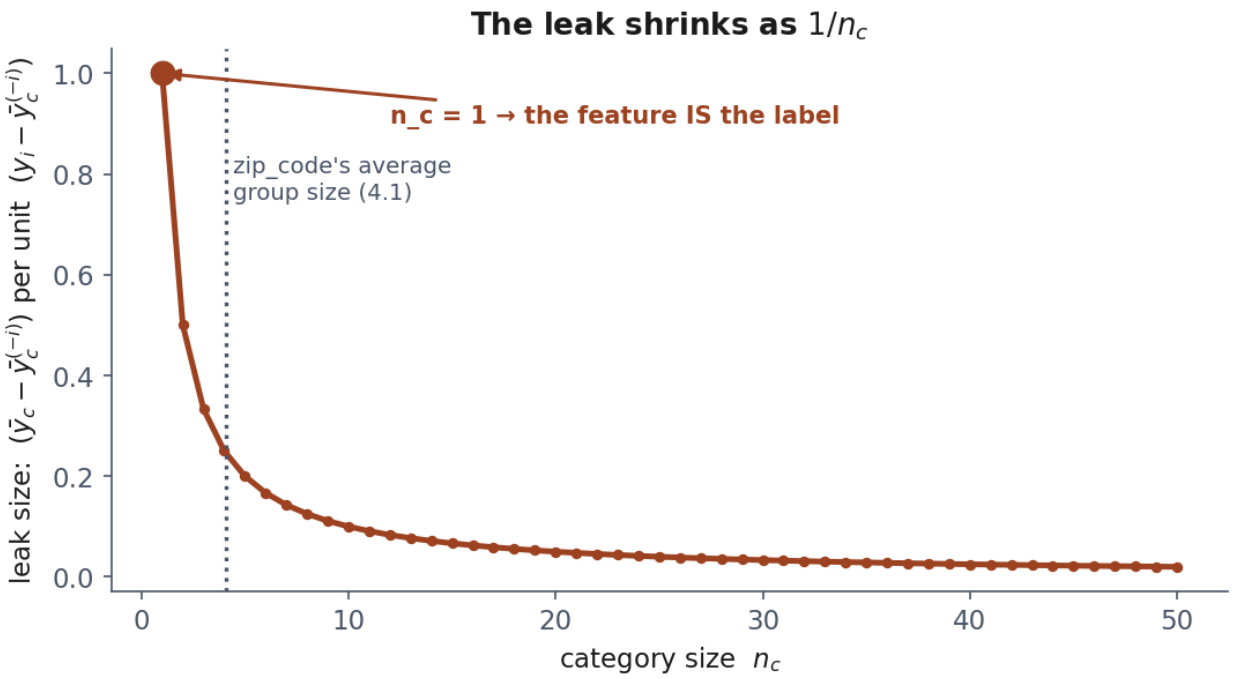
The algebra, for completeness. Nothing below is needed to use any of the above; it is the same statement in symbols. Write n_c for the number of rows in category c, $\bar{y}_c$ for the leaky average over all of them, and $\bar{y}_c^{(-i)}$ for the honest average with row i left out. The leaky average is a weighted blend of the honest one and the row's own label, and subtracting one from the other leaves

$$\bar{y}_c - \bar{y}_c^{(-i)} = \frac{y_i - \bar{y}_c^{(-i)}}{n_c}$$

which is the table: the pull is the row's own disagreement with its group, divided by the group's size. At $n_c = 1$ the right-hand side is undefined because there is no group left — and the encoding collapses to $\bar{y}_c = y_i$, the case notebook 3 prints.

One honest caveat. This describes the realistic bug: an encoding computed once over training and test data together. An encoding computed on the training fold alone is still slightly optimistic *for the training rows*, by the same arithmetic — but those values are only ever fitted to, never scored on, so that particular bias does not reach the test number. It matters more for models flexible enough to exploit it, such as Lesson 7's boosted trees, than for a single linear coefficient.

9.3 The rule, restated



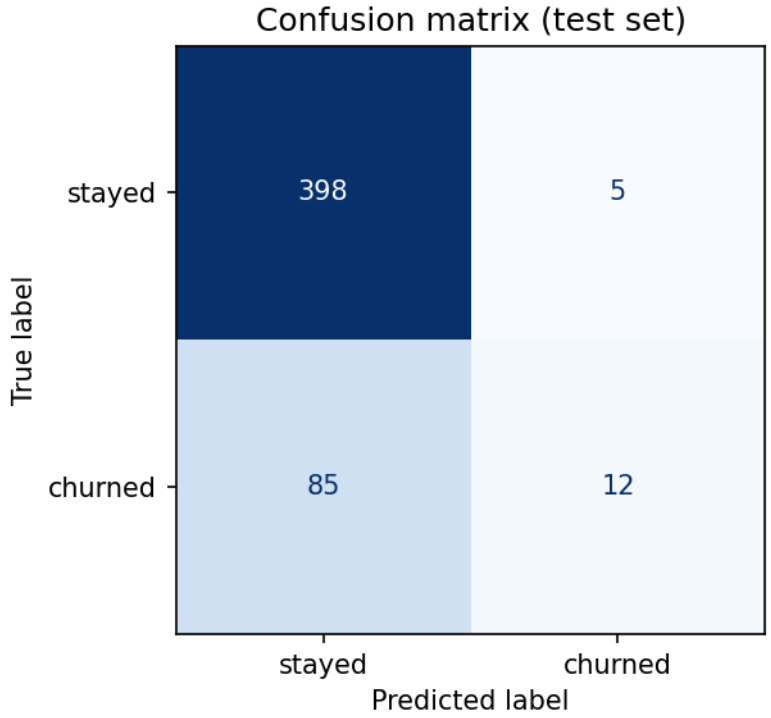
The leak is worst where the groups are smallest, falling as $1/n_c$ — which is also where a high-cardinality column keeps most of its categories.

Everything that learns from data — a mean, a median, a set of neighbours, a per-category target average, a set of bin edges — belongs inside the fold it is fitted on. This is not a longer list of special cases to remember; it is one test, applied to every preprocessing step in this lesson: *does fitting this*

step compute anything from the rows it is given? If yes, it goes inside the pipeline. Lesson 5 returns to leakage a third time, in the context of cross-validation, where the same test applies once more, this time to hyperparameter tuning.

---

10. Before the next lesson



The model built on the prepared data. Read the bottom-left cell: the churners it missed. Lesson 4 gives this picture its proper treatment.

1. Work through the three notebooks in order — 01 for exploration, missing values and outliers; 02 for scaling, encoding and the pipeline; 03 for the two leaks.
2. Take the quiz in `Quizzes/`.
3. **Complete the homework** in `Exercises/02_data_exploration_and_preparation.md`, due at the start of Lesson 3, Friday 9 October.

Notation used in this lesson

Symbol	Meaning
X, Y	a feature and the target treated as random variables (Sections 3–5); X is **also** the $m \times n$ design matrix in Section 7
$x^{(i)}$	the i-th example
m, n	number of examples, number of features
μ, σ	a feature's mean and standard deviation

Symbol	Meaning
$\bar{x}$	a sample mean
ρ	the correlation between two variables
p	the fraction of a column's values that are missing (Section 4)
$P(\cdot)$	a probability
k	the number of categories in a column (Section 6); **and**, in Section 5 only, the z-score cut-off, $k = 3$

Further reading

Resource	Type	Why read it
scikit-learn: `ColumnTransformer`	Official docs	The canonical reference for mixed numeric/categorical preprocessing
scikit-learn: `TargetEncoder` user guide	Official docs	How cross-fitting is implemented, and its parameters
Rubin, *Inference and Missing Data* (1976)	Paper	The original MCAR/MAR/MNAR taxonomy, still the standard reference
Kohavi & Provost, *Glossary of Terms* (1998), entry on "leakage"	Paper	An early, still-cited naming of the leakage problem this lesson extends
Micci-Barreca, *A Preprocessing Scheme for High-Cardinality Categorical Attributes* (2001)	Paper	The original target-encoding paper, including the shrinkage idea behind cross-fitting
Géron, *Hands-On Machine Learning*, ch. 2	Book	A complementary treatment of the full preparation pipeline, with a different worked example